THE PLACEMENT PLATFORM, ON A PHONE

prephasz mobile, role by role.

prephasz is a placement-preparation platform with four roles, 139 web pages and a live production backend running real college drives. The mobile app is not a port of that product. It is a deliberately narrow companion that does the handful of things a phone is genuinely better at — and routes everything else to the web, on purpose.

Same backend. Same APIs. No second source of truth. The server issues the attempt, the deadline, the questions and the grade; the app renders what it is told.

4
roles, all signing in from day one

75
modules inventoried across the platform

18
of them in the first release

139
pages in the web product today

0
new sources of truth

Narrow on purpose

Every module has a verdict: in v1, planned, later, or staying on the web. The module catalogue is the record of what was left out deliberately.

The server decides

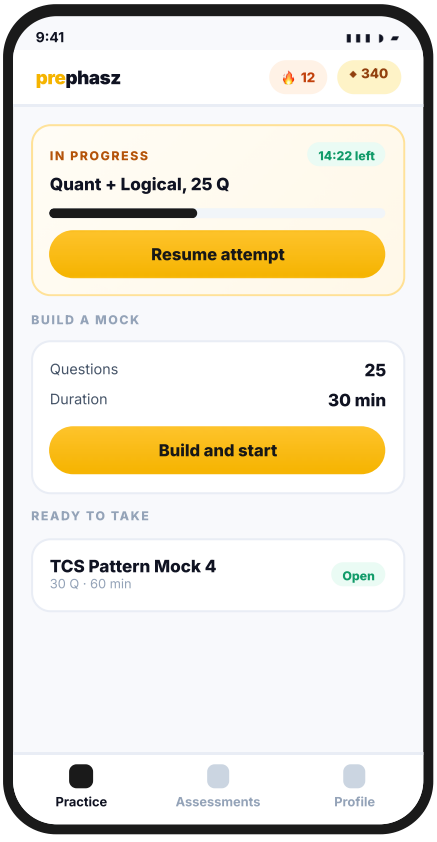
Timer, grading, entitlement and every gate are server-authoritative. The app computes no business value.

No camera, no checkout

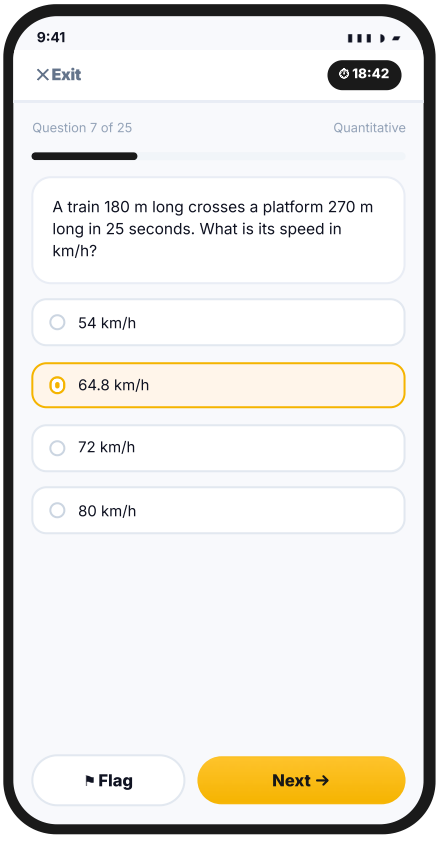
No proctoring and no purchase surface in v1 — the two decisions that remove most of the launch risk.

Stoppable in minutes

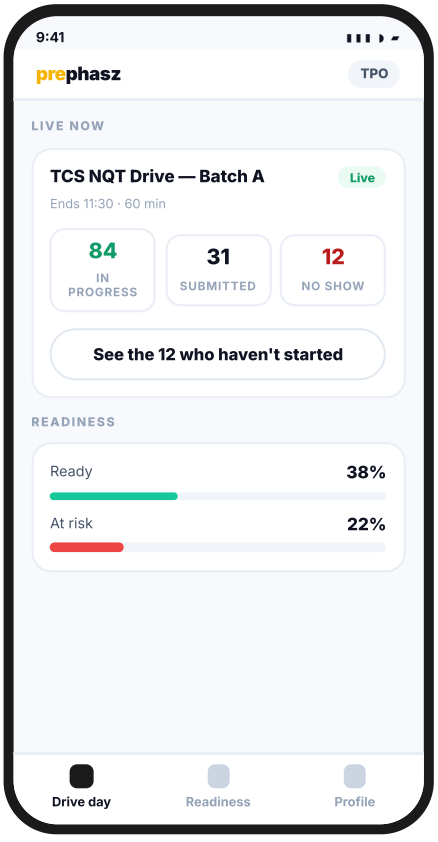
Remote config can raise the version floor or halt every build without waiting on a store review.



Practice
the student home



The runner
server-timed, offline-tolerant



Drive day
what a TPO needs standing in the lab

HOW IT WORKS

Four roles, three tab bars

A session resolves to a role, and the role decides which app renders. There is no separate admin build and no second login: the same binary serves a student, a placement officer and platform staff, with a different bottom tab bar and a different set of endpoints behind it. Authorisation is never the app's job — every screen sits behind the same backend guard the web uses.



What signing in actually resolves. The role is a claim on a server-signed token; the app reads it to pick a tab bar, which is a convenience, not a security boundary. A student who edited their own role claim would reach screens whose every endpoint still refuses them.

01

Student

The reason the app exists. Builds and sits timed mock assessments, sees upcoming drives, reads reports.

7 modules in v1

02

TPO

Placement officer at a partner college. Drive-day participation and cohort readiness, read-only.

2 modules in v1

03

Admin

prephasz operations. Watches a live drive and publishes results. The other 32 console pages stay on the web.

2 modules in v1

04

Super Admin

Same mobile surface as Admin in v1. Finance, audit and access control are desk work and stay there.

2 modules in v1

THE SESSION

The one thing that could not be ported

Web is cookie-only, and stays that way

The refresh token is an HttpOnly cookie precisely so page script can never read it. That posture is unchanged by anything the app does.

The app uses the Keychain instead

A native client has no cookie jar worth hanging a timed assessment on, so it receives the token in the response body and stores it in hardware-backed storage.

The guard fails closed

Native means the client header is present *and* neither Origin nor Referer is. Every browser sends one of those; a native fetch sends neither.

THE SPINE

Four decisions that shape everything

Settled by the owner before a line of app code was written. Each one removes work and risk rather than adding a feature, which is why they are at the front of this document rather than buried in an appendix.

D1 — Settled

The app sells nothing

Entitlement-aware but purchase-free. Where something is locked it says so plainly — no buy button, no price, no link to the web store on iOS. Students upgrade on the web and the app picks it up on next load.

Why: digital content sold outside IAP is the classic App Store §3.1.1 rejection, and it is discovered at review — after the whole project is built.

D2 — Settled

No proctoring, no coding

Proctored sittings and papers containing coding questions are visible but not startable, with an honest state. This removes the camera from the app entirely.

Why it is nearly free: the mock engine's default path is already unproctored, so the same server-timed, server-graded engine runs with no backend change at all.

D3 — Infrastructure

A dedicated API host

The app talks to its own subdomain pointing at the same load balancer, rather than routing every candidate's traffic through the web host's proxy.

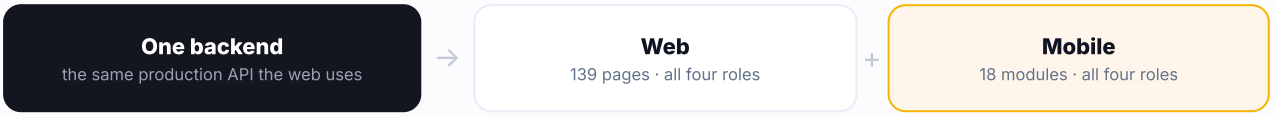
Why: the web calls its own origin on purpose, to keep the session cookie first-party. A native client has no such constraint and no reason to inherit the hop.

D4 — Settled

All four roles, one module each

Parity with 139 web pages is not a first release. A login that dead-ends for three of four roles is not a launch either. Each role gets the one thing a phone is genuinely good for.

Everything else is reachable as a single labelled "Open on web" row. It never pretends to be a feature that is missing.

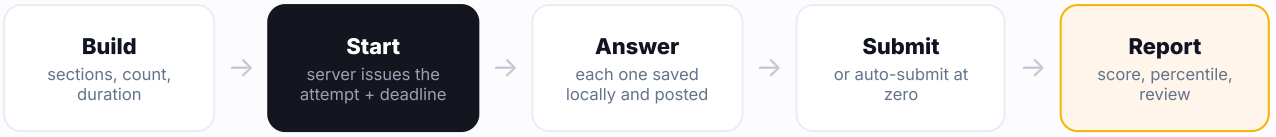


No business logic moves. Every value the app shows — score, percentile, XP, rank, time remaining, whether a surface is locked — is computed by the server and rendered by the client. There is nothing the app can get wrong that the web gets right.

THE CORE LOOP

One assessment, end to end

Everything in the student release serves this sequence. It is worth reading once, because four of the five hard requirements in the runner exist to protect a single step of it.



Start hands over the whole paper. One call returns the attempt id, the server-authoritative expiry and every question, plus any answers already saved. That is what makes a phone on campus wifi viable: the paper is in hand before the network gets a chance to be bad.

WHY RESUME IS A FEATURE, NOT AN ERROR PATH

A phone will interrupt the exam. The exam has to survive it.

A call arrives. The battery dies. Android kills the app to reclaim memory. On a laptop these are rare; on a phone they are Tuesday. The backend already solved this and the app inherits it for free: starting an attempt that is already open returns *that* attempt — same deadline, same saved answers — and deliberately skips the paywall and profile gates, so nobody mid-exam is ever stranded by a lapse in their plan.

The timer is the server's

The countdown derives from the issued expiry and re-anchors on every foreground and every successful request. Changing the device clock changes nothing.

Answers are durable

Each answer is written locally and posted; failures retry from a queue that survives a cold start. Sixty seconds in a lift loses nothing.

Nothing blocks the paper

Network, sync or telemetry failure degrades silently. The candidate keeps answering. This rule outranks every other behaviour in the app.

What the app refuses to start

A paper containing coding questions, and any proctored sitting. Both are shown with their window and duration, and an honest reason.

The refusal happens **before** the attempt is created. Starting and backing out would leave a live attempt behind and, on a drive, spend the one sitting a student gets.

What the phone is actually for

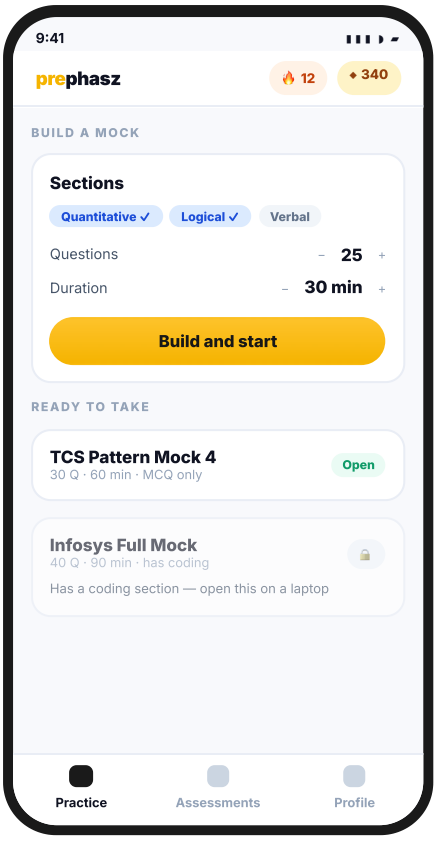
Practice between classes, and knowing when your drive is. High-stakes proctored sittings stay on a laptop with an invigilator.

Worth being precise about with colleges: the app is a practice and visibility surface, not an equivalent surface for an evaluated drive.

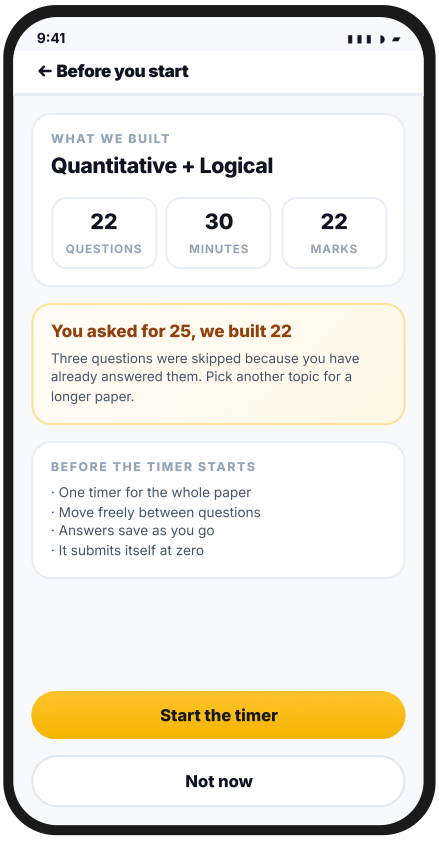
WORKSPACE 01 · STUDENT

Build a mock, and start it

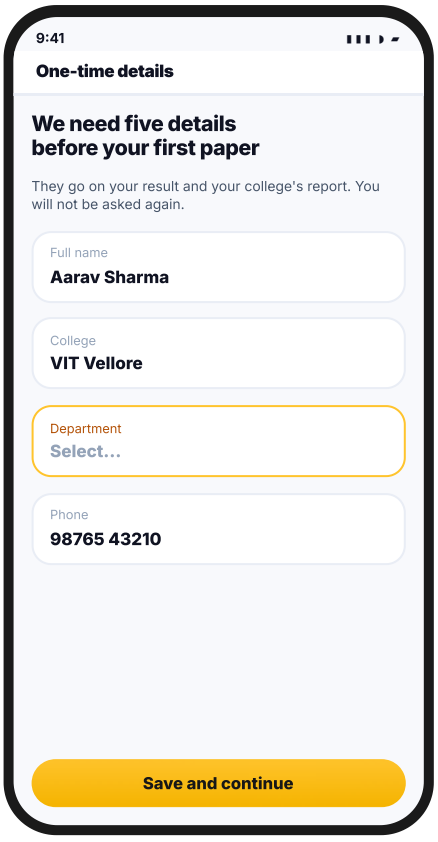
The student home opens on the one thing a student came to do. A live attempt always offers itself first; below that, a builder that assembles a paper from chosen sections and topics, then the catalogue.



Practice
builder, catalogue, and what it will not start



Pre-start
honest about what was actually assembled



Details gate
server-driven, only the missing fields

A live attempt wins

Anything already open is offered before anything new can be started.

Short papers are explained

Thin topics, a per-plan cap or questions you have already seen — said out loud, before the timer.

Coding refused up front

Checked on the paper's coding count before start, so backing out cannot spend a sitting.

Locked is honest

A locked surface says what is locked and that it is managed on the web. No price, no buy button.

REQUIREMENTS COVERED

Build, start and the gates in front of them

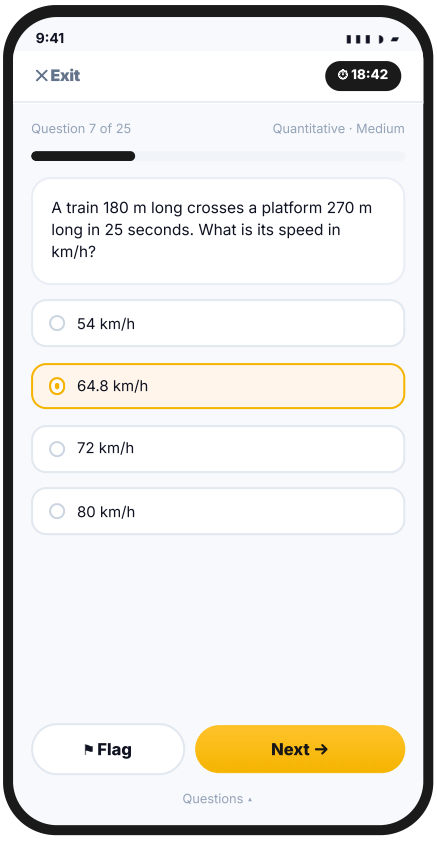
FR-2.1 · FR-2.2 · FR-2.3 · **FR-2.4** · FR-2.5 · FR-2.6 · FR-2.7 · FR-8.2

FR-2.4 is starred because it is the one that costs a student something when it is wrong: the coding check happens on the paper, before an attempt exists.

WORKSPACE 01 · STUDENT

The runner

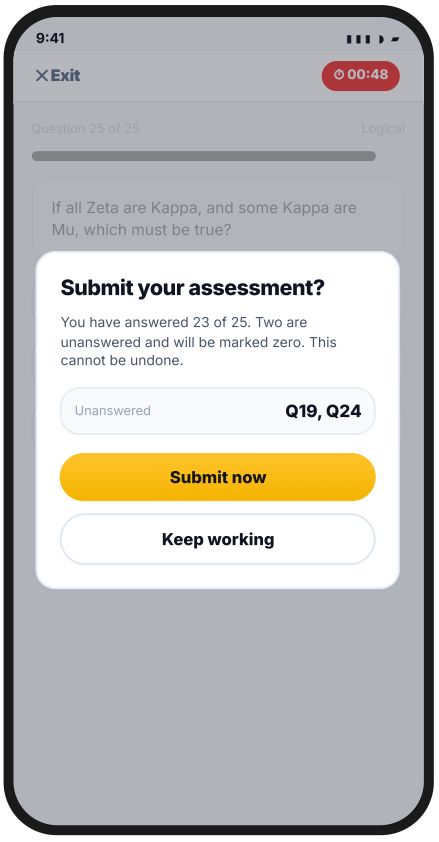
The screen that matters most, and the one with the least room for cleverness. One question at a time, large targets, a palette one tap away, and a clock that belongs to the server.



One question
single column, 44pt targets



Palette
answered, flagged, unseen — one tap away



Submit
never accidental, never ambiguous

The timer is the server's

Derived from the issued expiry, re-anchored on every foreground and successful request. A sleeping phone cannot lie about it.

Backgrounding is not an exit

A phone call mid-attempt returns to the same question. Only an explicit, confirmed exit leaves, and it leaves the attempt open.

Auto-submit at zero

At the deadline the attempt submits itself and the report opens. The student is never left wondering whether it counted.

REQUIREMENTS COVERED

The runner, and the five that are non-negotiable

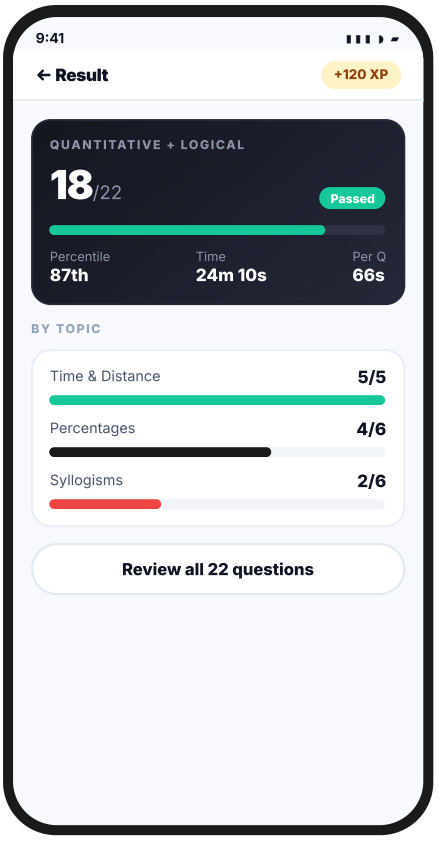
FR-3.1 FR-3.2 FR-3.3 FR-3.4 FR-3.5 FR-3.6 FR-3.7 FR-3.8 FR-3.9 FR-3.10 FR-3.11

Four of these exist only because this is a phone: a server-anchored clock, a durable answer queue, resume as a first-class path, and the rule that nothing in the app may block the paper.

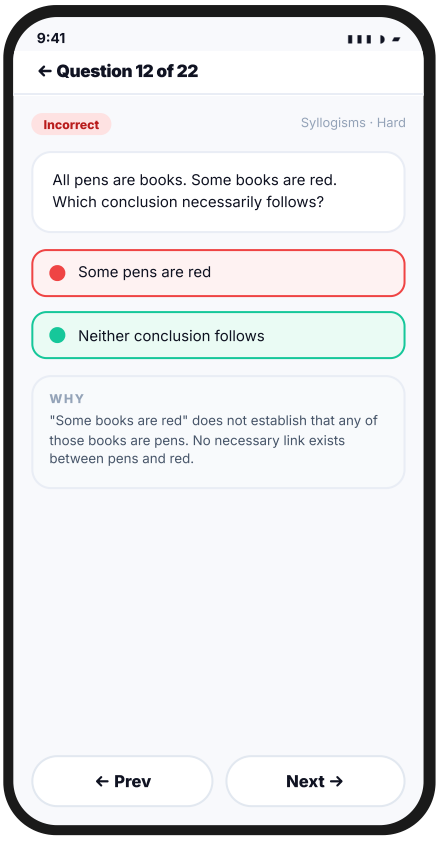
WORKSPACE 01 · STUDENT

The report, and the record

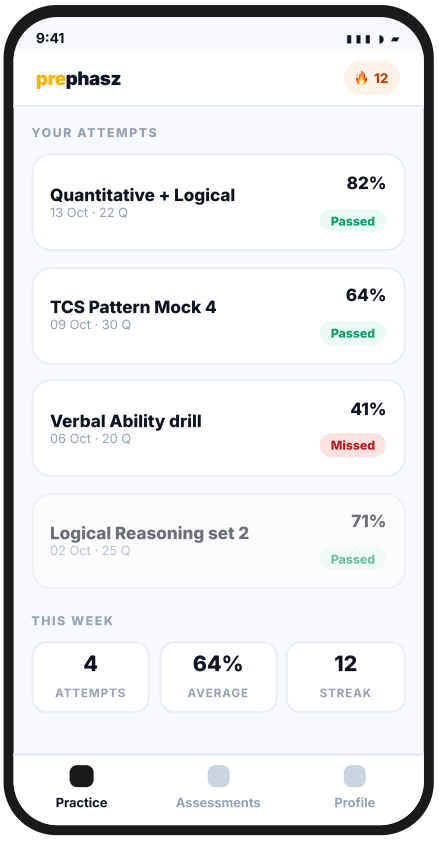
Every number here is computed by the server and rendered by the app. Score, percentage, percentile, pass verdict, topic breakdown and the XP the attempt earned all arrive as values, never as formulas.



Report
server-computed, top to bottom



Review
your answer, the right one, and why



History
every finalised attempt, newest first

Rendered, never computed

Percentile, pass verdict and XP arrive as values. The app has no formula to get wrong.

Full answer review

Every question, your answer, the correct one, and the explanation — the same content the web shows.

Re-openable

Any finalised attempt reopens its full report from history. Nothing is only available once.

Topic breakdown

Where the marks went, by topic, so the next practice session has an obvious subject.

REQUIREMENTS COVERED

Results and history

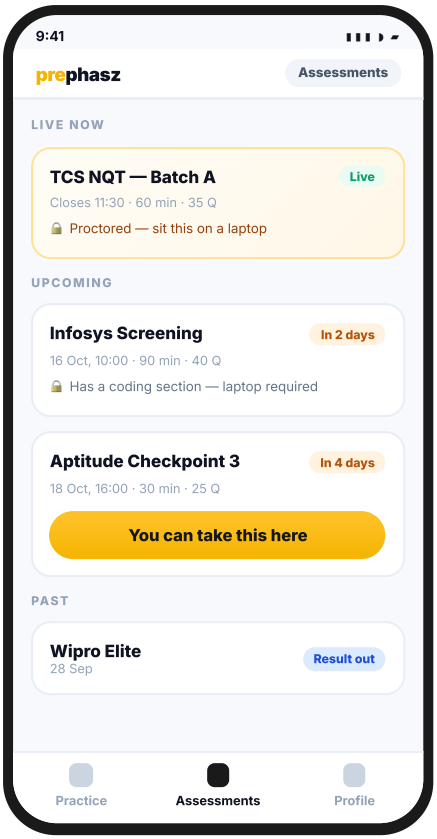
FR-4.1 FR-4.2 FR-4.3 FR-4.4 FR-4.5

Every value on these screens is server-computed. The app has no scoring, percentile or XP logic to drift out of step with the web.

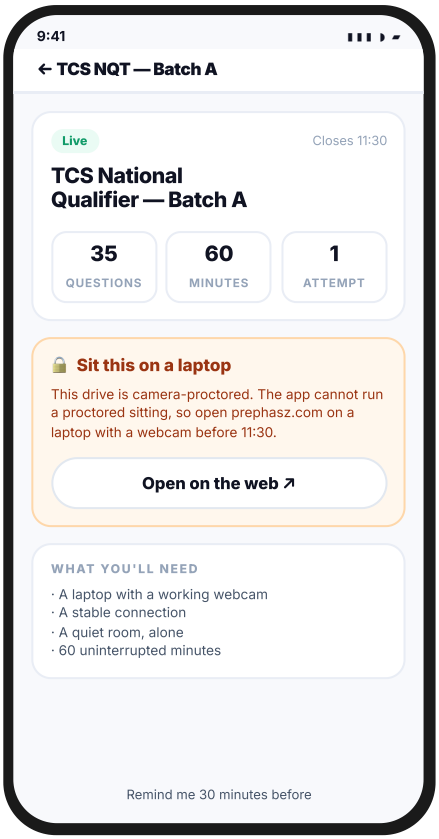
WORKSPACE 01 · STUDENT

Drives, visible but not sat

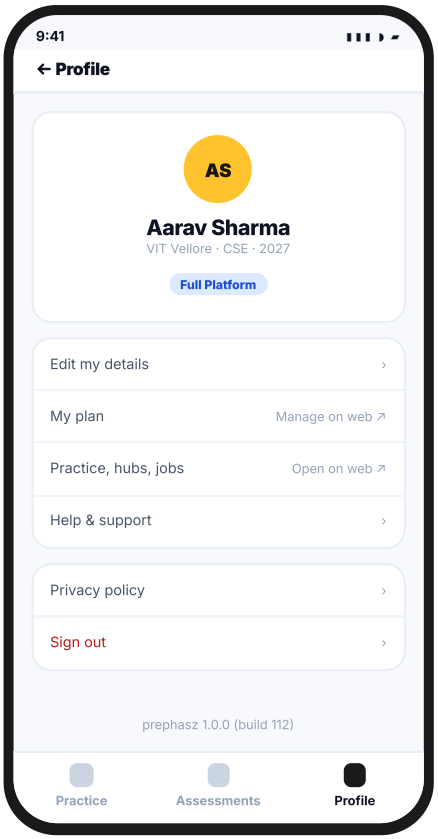
A college drive is the highest-stakes thing a student does on this platform, and in v1 the app does not run the proctored ones. It does something quieter and still valuable: it puts the schedule on the device the student actually carries, so nobody misses a sitting because it was only ever on a laptop.



Assessments
the schedule, and what can be sat here



A proctored drive
a reason, and a route, not a dead end



Profile
where "open on web" lives, labelled

The window comes from the sitting

A row's end time is read from the drive's real closing time, never inferred from start plus duration. That exact inference once made a live drive vanish from students' lists.

A refusal explains itself

Proctored or coding papers show why, what is needed, and a route to the web. A locked row is never just greyed out.

Push closes the loop

In the release after launch, "your drive starts in 30 minutes" is the single highest-value notification this product can send.

REQUIREMENTS COVERED

Drives, read-only, and the profile

`FR-5.1` `FR-5.2` `FR-5.3` `FR-5.4` `FR-5.5` `FR-5.6` `FR-8.1` `FR-8.3` `FR-8.5`

FR-5.2 is starred for history: reading a window as start plus duration, rather than the sitting's real closing time, once made a live drive disappear from students' lists mid-drive.

THE QUESTION EVERYONE ASKS FIRST

Where is the dashboard?

It is the densest page in the product — eighteen panels on the web — and it is deliberately not in the first release. Not because a dashboard is unimportant, but because almost every panel on it is a window into a module the app does not have yet. A v1 dashboard would be a page of links to things the app cannot do.

What the web dashboard carries

An AI briefing hero, a live-assessment banner, headline stats, a readiness panel, the daily quest, company cards, quick aptitude, the daily challenge, coding practice, challenges, the practice hub, an adaptive skill profile, and a right rail of deadlines and company readiness. Eighteen panels.

What survives without its module

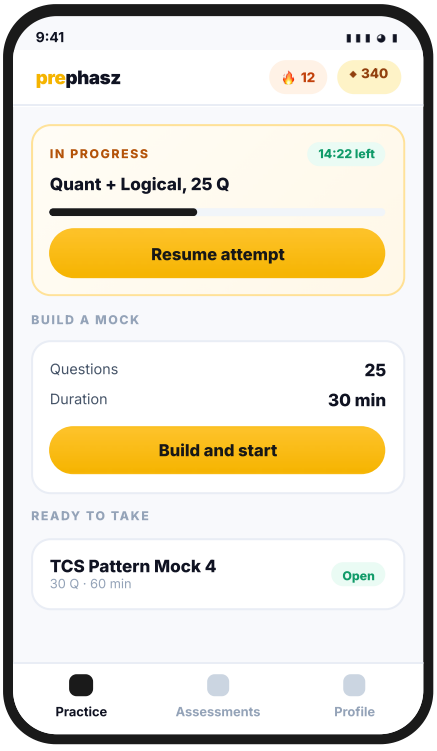
Three things. The streak and coin chips. The live-attempt banner. The practice entry point. Everything else needs practice, hubs, jobs, coding, challenges or gamification to exist first.

So in v1 the Practice tab is the home — it already carries all three, and it opens on the thing a student came to do rather than on a summary of it.

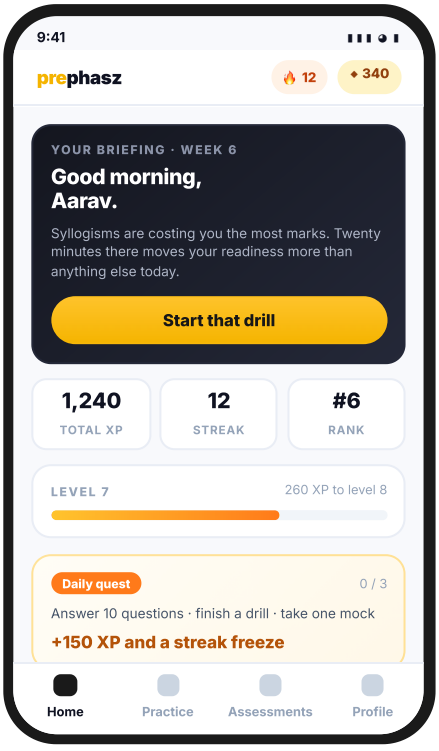
When the real one arrives

In the release after launch, alongside performance, calendar, practice and the hubs. By then its panels have something to point at, and the design below is what it becomes.

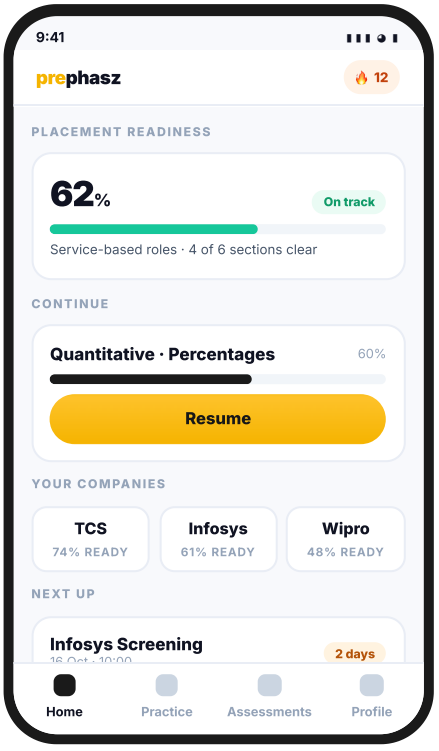
Drawn now rather than later because it shares components with the v1 home, and building those twice would be the expensive way round.



v1 · the home
no summary layer, just the work



P2 · the dashboard
briefing, standing, the day's quest



P2 · continued
readiness, continue, companies, deadlines

WHY THIS IS P2, PANEL BY PANEL

The dashboard question

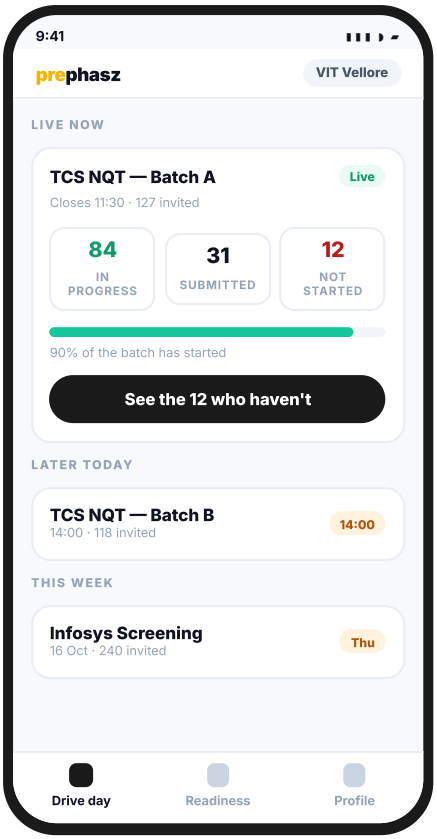
`streak + coins` `live attempt` `practice entry` `briefing` `readiness` `daily quest` `continue` `companies` `coding` `challenges` `skill profile` `deadlines` `participation` `calibration` `quick aptitude` `subscription`

The three highlighted panels work with only the mock engine behind them, and all three already live on the Practice tab. The other thirteen each need a module that is not in the first release — which is the argument: a dashboard is a window onto a product, and in v1 there is not yet enough product behind the glass.

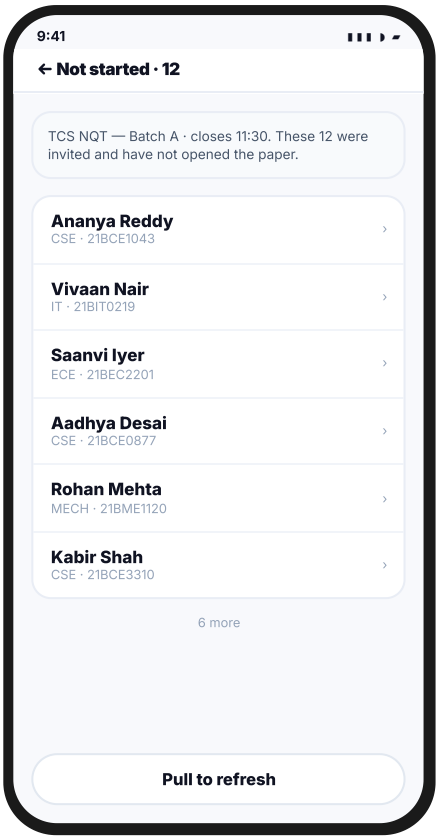
WORKSPACE 02 · TPO

Drive day, from the lab floor

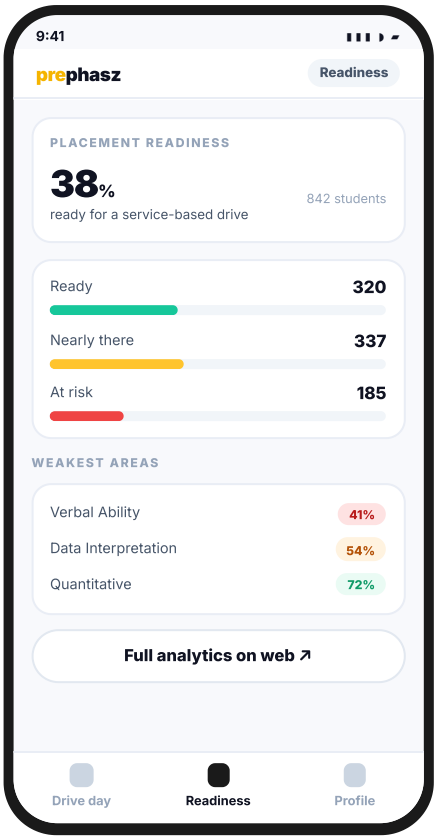
A placement officer has sixteen pages on the web and needs roughly two of them while standing in a room with ninety students. The question is never "what is our six-month skill-gap trend"; it is "who has not started yet, and what is their name".



Drive day
the number that matters, first



By name
enough to walk the room and find them



Readiness
read-only, server-computed

Read-only on purpose

Scheduling a drive, inviting a cohort and exporting a report are desk tasks. They stay on the web where they are better.

Refreshes on foreground

Coming back to the app during a live sitting shows the current state, not whatever was true when it was last opened.

Named, not counted

A count tells a TPO there is a problem. A list of names lets them solve it before the window closes.

One route out

Everything else is a single labelled "Open TPO console" action rather than fourteen half-built screens.

REQUIREMENTS COVERED — TPO drive day

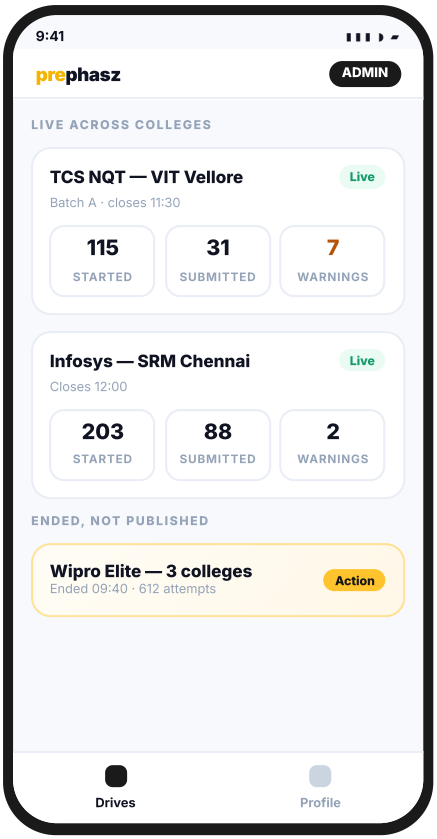
`FR-6.1` `FR-6.2` `FR-6.3` `FR-6.4` `FR-6.5` `FR-0.1` `FR-0.3`

Read-only by design. Every write a TPO performs — scheduling, inviting, exporting — is a desk task and stays on the web, reachable through one labelled action.

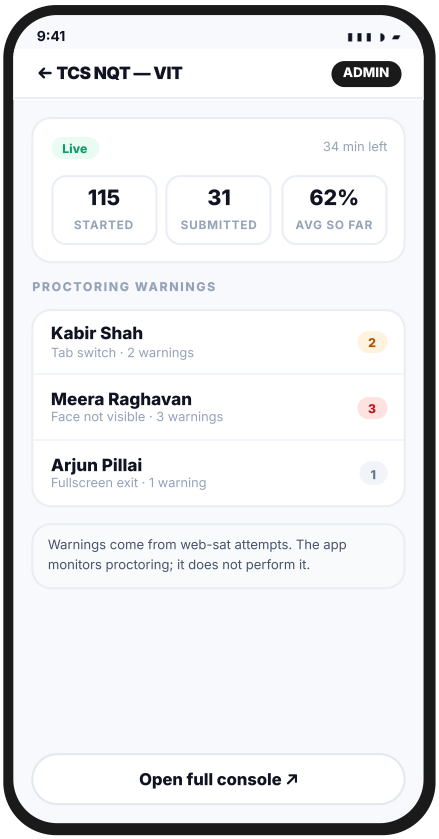
WORKSPACE 03 · ADMIN & SUPER ADMIN

Two consoles, one mobile job

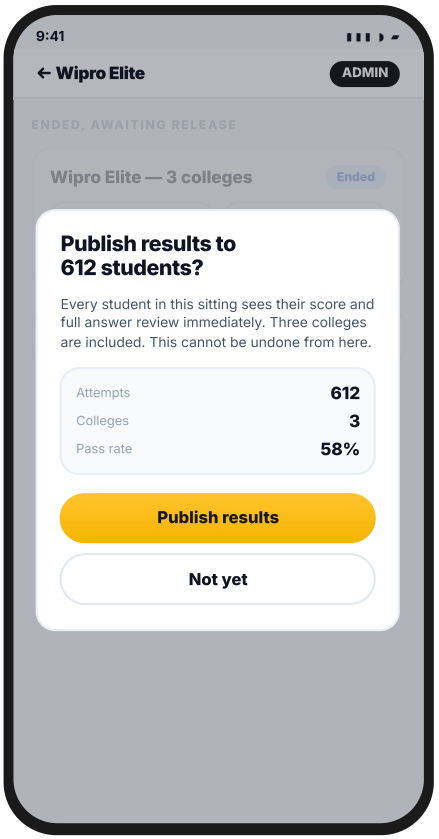
The admin and super-admin consoles are 67 pages of dense tables, CSV exports and large-screen charts. Almost all of it is desk work, and a phone version would be worse at the job than the web one that already exists. Exactly one slice is genuinely mobile: watching a drive while it runs, and publishing the result when it ends.



Drive monitor
every live sitting, one screen



A live sitting
warnings as they land



Publish
confirmed, because 612 people see it at once

Monitors, never proctors

The warnings shown here were raised by web-sat attempts. The app has no camera and takes no part in proctoring.

One write action

Publishing results is the only thing this surface changes, and it is confirmed first — it is visible to every student in the sitting at once.

32 pages stay put

Catalogue, users, colleges, finance, marketing, audit. All of it is better on a 27-inch screen and none of it is urgent on a phone.

REQUIREMENTS COVERED

The live drive monitor

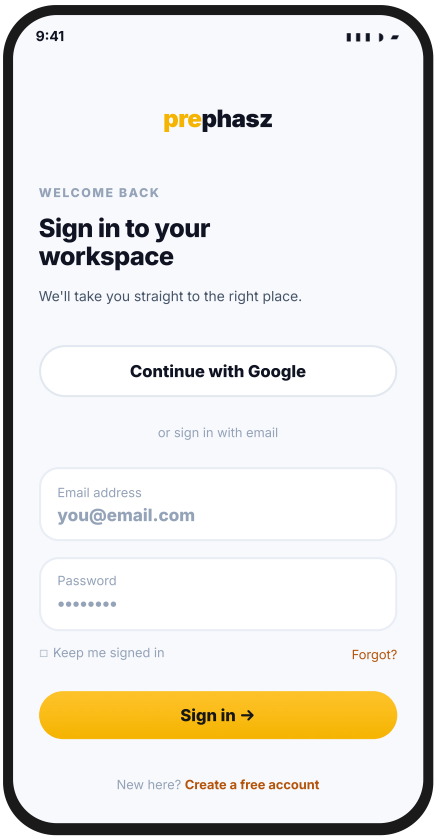
FR-7.1 · FR-7.2 · FR-7.3 · **FR-7.4** · FR-7.5 · FR-0.1 · FR-0.3

FR-7.4 is the only write in the whole admin surface, and it is confirmed first: publishing makes results visible to every student in the sitting at the same moment.

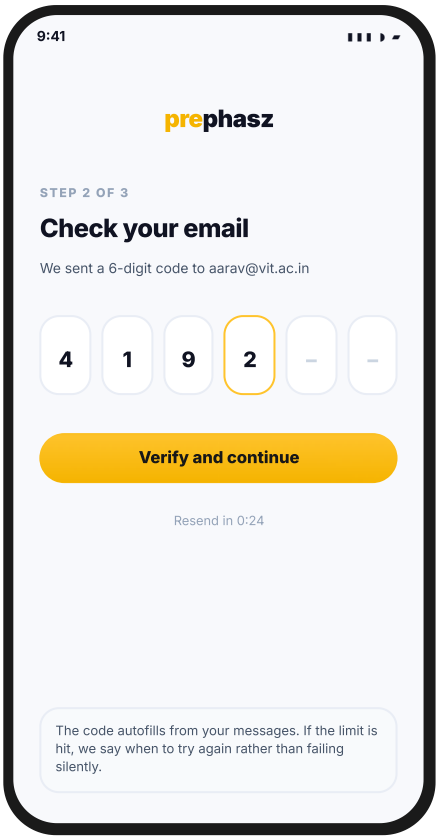
EVERY ROLE STARTS HERE

Sign in, and the three-step signup

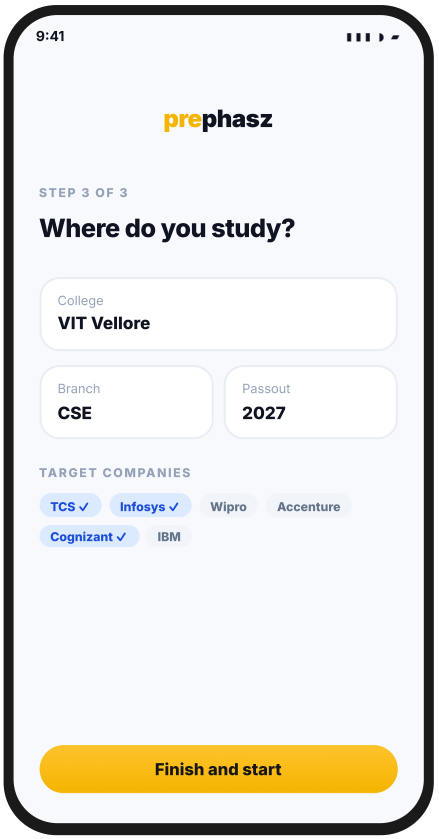
All seven authentication surfaces are in the first release, including the one most mobile apps skip: the set-password flow for a student a college bulk-invited. Colleges invite in batches of hundreds, and those students have to be able to activate on the phone in their hand.



Sign in
one screen, four roles behind it



Verify
OTP autofill, honest rate limits



Onboarding
college, branch, targets

Survives a force-quit

Kill the app, restart the phone, come back a day later — still signed in. The session lives in hardware-backed storage, not a cookie jar.

Native Google sign-in

The platform SDK, not a web view. A cancelled flow returns to the form with no error state.

Invited students included

Set-password is in v1 precisely because colleges onboard in bulk and those students arrive on a phone.

Suspended is explained

A suspended account cannot refresh into a session. The app says so as an account message, not a network failure.

REQUIREMENTS COVERED

Every authentication surface

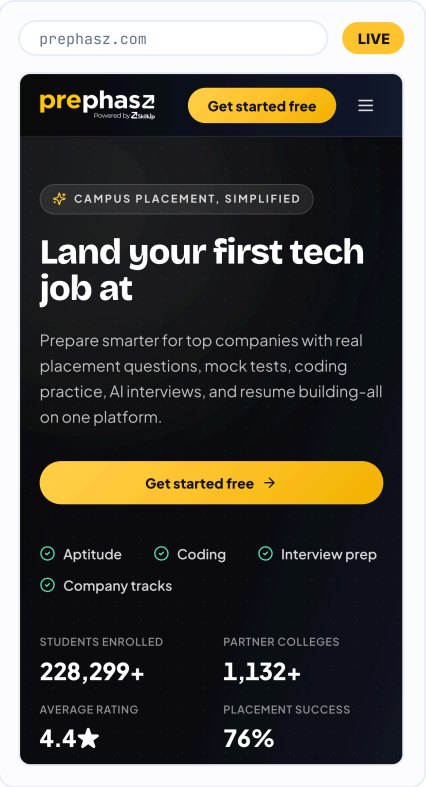
`FR-1.1` `FR-1.2` `FR-1.3` `FR-1.4` `FR-1.5` `FR-1.6` **`FR-1.7`** `FR-1.8` `FR-1.9` `FR-1.10` `FR-0.4`

FR-1.7 is the one to test hardest: a session that does not survive a force-quit turns every cold start into a login screen, which is how a practice habit dies.

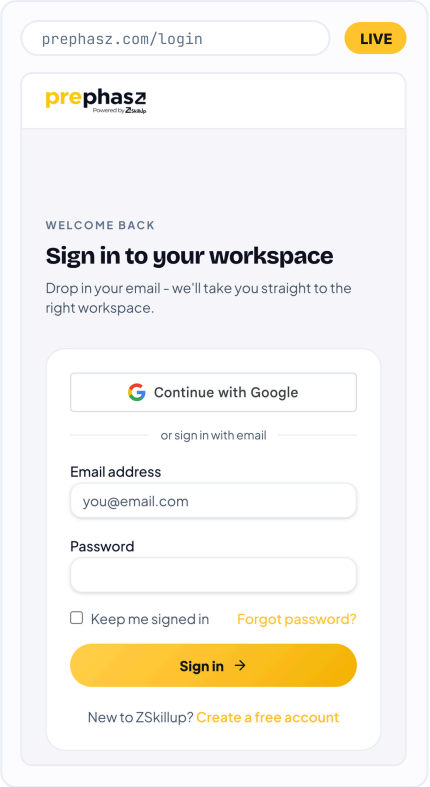
TODAY, FOR COMPARISON

The web at phone width

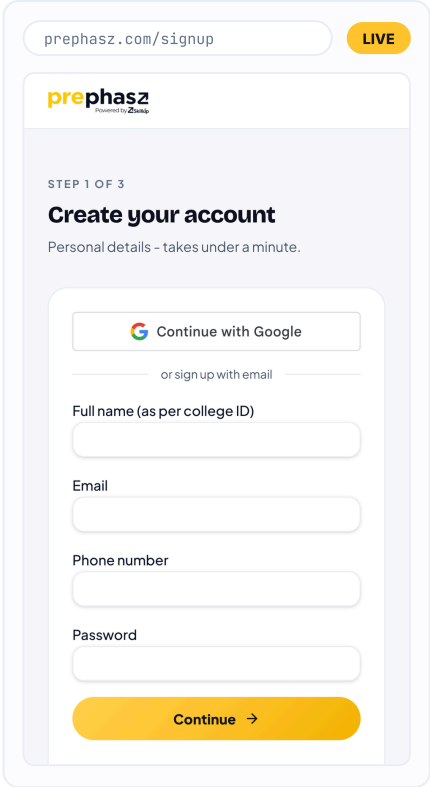
Captured from the live site at 390 × 844 on 13 October 2026. These are real pages, rendered as a phone renders them now — the starting point the app improves on, not a design target. Marketing pages stay on the web; the store listing is the app's front door.



Home
stays on the web — v1 has no marketing surface



Login
the app reproduces this with native keyboard and Google sign-in



Signup, step 1 of 3
the same three steps, with OTP autofill

Why the authenticated screens are not here

Production needs credentials this document's author does not hold, and the local environment has no seeded data. Empty-state screenshots of a dashboard with no students in it would be misleading in a requirements document, so they are absent rather than faked.

Two ways to add them before design sign-off: a read-only production student and TPO login, or a seeded local stack. **Worth doing.**

One thing capturing turned up

The public job board at `/jobs` returns HTTP 200, but a browser lands on `/login`. The board sits inside the authenticated route group, while only individual job pages are public.

Possibly intended. Worth confirming, because the public board is described as a marketing surface and appears in the sitemap.

NOTHING LEFT OUT

Every module, by role

The complete inventory of what the platform does today, with a verdict on each. This is the point of the document: the record of what was consciously left out, rather than forgotten. **V1** first release **P2** planned next **P3** later **WEB** stays on the web

Student

32 · 7 in v1

Mock Assessment builder	V1
Mock runner	V1
Mock report	V1
Mock history	V1
Assessments / drives	V1 read-only
Profile	V1
Help & support	V1
Dashboard	P2
Study Plan	P2
Performance	P2
Leaderboard	P2
My Calendar	P2
Assessment leaderboard	P2
Adaptive practice	P2
Non-adaptive practice	P2
Practice review	P2
Adaptive quiz runner	P2
Company Hubs (7 tabs)	P2
Company prep & PYQs	P2
Sectional Hubs	P2
Job board	P2
My applications	P2
Certificates	P2
My Learning	P2
Recommendations	P2
Community	P3
Live Sessions	P3
Mock Interview	P3
How XP works	P3
Resume Builder	WEB
Coding practice	WEB
Plans, upgrade, cart	WEB

TPO

15 · 2 in v1

Drive day — participation	V1
Placement Readiness	V1 read-only
Executive Dashboard	P2
Student Analytics	P2
Company Readiness	P2
Skill Gap Analytics	P2
Student Management	P2
Coding Analytics	WEB
Interview Analytics	WEB
Assessment Center	WEB
Cohorts	WEB
Invitations	WEB
Reports & Exports	WEB
Subscription & billing	WEB
Settings	WEB

Auth & public

14 · 7 in v1

Login, Google, signup ×3, reset, set-password	V1
Public job detail	P2
Roadmap	P3
Home, Prepare, About, Blog, Verify	WEB

Admin & Super Admin

14 · 2 in v1

Live drive monitor	V1
Publish results	V1
Dashboard & analytics	P2
Support inbox	P2
Broadcasts	P3
Users, colleges, cohorts	WEB
Catalogue (9 surfaces)	WEB
Commerce (5 surfaces)	WEB
Marketing (4 surfaces)	WEB
Jobs administration	WEB
Feature locks, calibration	WEB
Audit, user sheet, reports	WEB
Live sessions, WhatsApp	WEB
Mobile remote config *(new)*	WEB

HOW TO READ THIS

What these screens are, and are not

Accuracy matters more than polish in a document that gets signed off, so a few things are worth stating plainly.

- **The phone screens are mockups, not the live product.** The app does not exist yet; this document is part of deciding what it should be. The mockups are drawn to the platform's own design tokens — the same yellow, the same near-black ink, the same card and pill treatments — so they read as prephasz rather than as a generic deck. They are structural: what is on the screen and in what order. They are not a design specification.
- **The web screenshots are real.** Captured from the live site at phone width on 13 October 2026, shown as they loaded. Authenticated screens are deliberately absent rather than faked from an empty database.
- **The numbers come from the codebase.** 139 web pages, 75 modules, four roles and the module verdicts were read out of the repository, not estimated. Where something was inferred rather than verified it is marked as an assumption in the requirements document.
- **Sample data is invented.** Student names, scores, college names and drive titles are illustrative. No real student record appears anywhere in this document.
- **Scope is settled; the rest is not.** The four decisions are agreed. Screen layouts, copy and visual design are open, and the design system is written before any screen is built.

STILL OPEN

Five things that need an owner

Play Console account type

Organisation accounts go straight to production review. A personal account needs roughly twelve testers over fourteen days first — two to three weeks of calendar, and the single largest unknown in the launch date.

Session length on mobile

The refresh window is seven days and global. Leaving it means a student signs in weekly; raising it changes the web's posture too.

Screenshots of authenticated screens

A read-only production login, or a seeded local stack. Needed before design sign-off.

Minimum OS versions

To be set against the real student device mix rather than a guess. Low-end Android is the design target.

WHAT HAPPENS NEXT

The build has already started.

Three pull requests are open against the production repositories. None of them changes a single thing about how the web product behaves — that was the constraint, and the tests assert it.

backend · open

The app holds its own session

A native client receives the refresh token in the body and keeps it in the Keychain. The guard fails closed: a browser can never opt in, even by spoofing the header. **17 tests**, all of the important ones negative.

backend · open

Remote config and a coding count

A version floor, an outage stop and per-surface flags — the lever that stops a bad build without a store review. Plus the field that lets the app refuse a coding paper before it starts one.

frontend · open

The plan, the log and this document

A costed engineering plan, a build log written to be picked up cold, and the requirements document behind these pages.

21
working days of build

4
sprints, A through D

11 Nov
submitted to both stores

Late Nov
live, if the accounts are ready

1,685
backend tests passing

The critical path is not the code

At this pace the build finishes in about four weeks. Apple's organisation enrolment takes two to four. They now finish together, which means either one can be the launch date — and only one of them is under our control. Start both developer enrolments today, confirm the Play account type, and get a cheap Android into the room.